

India Tour Planner AI

Comprehensive Architecture and System Documentation Overview

1. Executive Summary

The **India Tour Planner AI** is a state-of-the-art web application designed to generate highly customized, day-by-day travel itineraries within India. The app uses a **RAG (Retrieval-Augmented Generation)** system combined with vector semantic search and cloud-hosted LLM reasoning to construct context-rich, budget-friendly trip schedules. The project is built using a professional, modern dual-stack split architecture with separate frontend and backend layers to maximize performance, security, and scalability.

2. Programming Languages & Tech Stack Overview

The application relies on three distinct languages and specialized frameworks, each dedicated to a specific area of the system:

Language / Tool	Framework / System	Primary Purpose & Work
JavaScript	React (Vite)	Builds the client-side user interface, handles browser routing, page views, and captures form selections.
Python	FastAPI	Powers the backend API server, performs RAG database searches, and coordinates the Groq AI integrations.
SQL	Turso Cloud (libSQL)	Creates, updates, and queries relational database tables (users, trips, cache) stored permanently in the cloud.
Firebase SDK	Auth Client SDK	Handles third-party authentication (Google Login, Phone SMS OTP) and stores local session tokens in the browser.

3. Detail Breakdown: How Each Component Works

3.1 React Frontend (Client Layer)

The frontend is built as a single-page application (SPA) using React, styled with vanilla TailwindCSS-like glassmorphic theme elements. Its operations include:

- **Client Router:** Directs page loading (Dashboard, Login, Profile) without full page reloads, providing an instant native-app feel.
- **Firebase Session Management:** Configured with local persistence so user credentials survive browser restarts. Operates in simulated mode if offline keys are provided.
- **Itinerary Export:** Generates customized PDF downloads of trip plans directly in the browser.

3.2 Python FastAPI (Server & API Layer)

The backend is a high-performance Python application built on FastAPI. It contains:

- **FastAPI Router:** Defines RESTful API endpoints called by the React client (e.g. login, signup, generating plan).
- **Token Validator:** Uses secure middleware to verify Google/Phone JWT tokens against Google's OAuth APIs.
- **Password Security:** Uses sha256 secure salting/hashing algorithms to store local email credentials safely.

3.3 Turso Cloud Database (Storage Layer)

We use a hosted cloud database powered by **Turso DB**. This is critical for data durability because:

- **Data Durability:** Traditional servers like Render Free Tier delete their local disks regularly. Turso saves data in the cloud, so your users and itineraries are never lost.
- **SQLite Schema:** Since Turso runs libSQL (SQLite-compatible), we keep a lightweight, fast, SQL-queried database without complex setup.
- **Python PEP 249 Adapter:** The backend implements a custom row-factory adapter to convert database rows to dictionaries, maintaining 100% codebase compatibility.

3.4 RAG Search & Groq LLM (Intelligence Layer)

When a trip is requested, the system uses a RAG pipeline rather than relying solely on raw AI knowledge:

1. **Retrieval:** The app searches the vector store index (vector_store.json) to fetch verified, real tourism records for the destination city.
2. **Augmentation:** These records (hotels, restaurants, attractions) are dynamically injected into the LLM system prompt.
3. **Generation:** The payload is pushed to the high-speed **Groq completions API** using the flagship openai/gpt-oss-120b model to draft the final markdown tour plan.

4. Step-by-Step Trip Generation Lifecycle

Below is the exact sequence of events that takes place when a logged-in user requests a new travel plan:

1. Input Submission

User submits destinations, days, and budget on the React interface.

2. API Request

React sends a JSON payload to the Python FastAPI `/api/plan-trip` endpoint.

3. RAG Search

Python scans `vector_store.json` for the cities and retrieves matching tourism vectors.

4. LLM synthesis

FastAPI formats the vectors, calls Groq (`openai/gpt-oss-120b`), and receives the itinerary.

5. Database Save

Python saves the generated plan into the Turso Cloud `trips` table for that user.

6. Rendering

FastAPI returns the markdown text, and React parses and displays it as a beautiful interactive timeline.